

Московский авиационный институт (национальный исследовательский университет)

Институт «Компьютерные науки и прикладная математика»

Кафедра «Вычислительная математика и программирование»

Лабораторная работа №1

по курсу «Дискретный анализ»

на тему «Сортировки за линейное время»

Студент: Строков Георгий Владимирович

Группа: М8О-208БВ-24

Преподаватель: Макаров Н.К.

Проверил: Сахарин Н.А.

Дата: 6 марта 2026 г.

Оценка: _____

Подпись: _____

Москва, 2026

1 Постановка задачи

Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа указанным алгоритмом сортировки за линейное время и вывод отсортированной последовательности. Алгоритм сортировки: поразрядная сортировка.

2 Формат ввода

На каждой непустой строке входного файла располагается пара «ключ-значение», в которой ключ указан согласно заданию, затем следует знак табуляции и указано соответствующее значение. Каждая строка содержит:

- Тип ключа: даты в формате DD.MM.YYYY.
- Тип значения: числа от 0 до $2^{64}-1$.

Пары разделены пробельными символами.

3 Формат вывода

Выходные данные состоят из тех же строк, что и входные, за исключением пустых и порядка следования. Порядок следования определяется алгоритмом сортировки, согласно условию задания.

4 Метод решения (Алгоритм)

Используется алгоритм **Radix sort** (поразрядная сортировка):

1. Программа сортируется по дням, по месяцам, далее - по годам.
2. Ищется число k - максимальное число сортируемых данных.
3. Создается массив `Array` размера $k + 1$, заполненный нулями.
4. Алгоритм проходит по исходному массиву и для каждого целочисленного элемента увеличивает значение в массиве `Array`, ключом которого является целочисленный элемент.
5. Алгоритм проходит по массиву `Array` и для каждого элемента строит префиксную сумму (`Array[i] += Array[i - 1]`).
6. Алгоритм проходит с конца по исходному массиву, получает ключ `key`, который равен элементу исходного массива. Далее в выходной массив по ключу (`key - 1`) ставится элемент исходного массива. Значение массива `Array[key]` уменьшается на единицу.

5 Исходный код

```
1 #include <iostream>
2 #include <c std def>
3 #include <c std int>
4
5
6 struct Item {
7     uint8_t day;
8     uint8_t month;
9     uint16_t year;
10    uint32_t pos;
11    uint32_t len;
12 };
13
14
15 template <typename T>
16 class MyVector {
17 private:
18     T* data_;
19     std :: size _t size _;
20     std :: size _t capacity _;
21
22 public:
23     MyVector()
24     : data_{new T[10]() }
25     , size_{0}
26     , capacity_{10}
27     {}
28
29     explicit MyVector( std :: size _t n)
30     : data_{new T[n]() }
31     , size_{n}
32     , capacity_{n}
33     {}
34
35     MyVector( const MyVector& other)
36     : data_{new T[other.capacity_] }
37     , size_{other.size _}
38     , capacity_{other.capacity _}
39     {
40         for ( std :: size _t i = 0; i != other.size _; ++i) {
41             data_[i] = other.data_[i];
```

```

42     }
43 }
44
45 MyVector(MyVector&& other) noexcept
46 : data_{other.data_}
47 ,   size_{other.size_}
48 ,   capacity_{other.capacity_}
49 {
50     other.data_ = nullptr;
51     other.size_ = 0;
52     other.capacity_ = 0;
53 }
54
55 MyVector() noexcept {
56     delete[] data_;
57     size_ = 0;
58     capacity_ = 0;
59 }
60
61 MyVector& operator=(const MyVector& other) {
62     if (this == &other) {
63         return *this;
64     }
65
66     delete[] data_;
67     data_ = new T[other.capacity_];
68     size_ = other.size_;
69     capacity_ = other.capacity_;
70     for (std::size_t i = 0; i != other.size_; ++i) {
71         data_[i] = other.data_[i];
72     }
73
74     return *this;
75 }
76
77 MyVector& operator=(MyVector&& other) noexcept {
78     data_ = std::move(other.data_);
79     size_ = other.size_;
80     capacity_ = other.capacity_;
81     other.data_ = nullptr;
82     other.size_ = 0;
83     other.capacity_ = 0;
84

```

```

85         return *this;
86     }
87
88     T& operator [] (std :: size _t index) { return data_[index]; }
89     const T& operator [] (std :: size _t index) const { return data_[index]; }
90
91     std :: size _t size () const { return size_; }
92     std :: size _t capacity () const { return capacity_; }
93
94     void push_back (const T& value) {
95         if ( size _ == capacity_ ) {
96             capacity_ = (capacity_ == 0 ? 1 : capacity_ * 2);
97             T* new_data = new T[capacity_];
98             for ( size _t i = 0; i != size _; ++i) {
99                 new_data[i] = std :: move(data_[i]);
100             }
101
102             delete [] data_;
103             data_ = new_data;
104         }
105
106         data_[ size _++] = value;
107     }
108
109     T* begin () { return data_; }
110     const T* begin () const { return data_; }
111     T* end () { return data_ + size _; }
112     const T* end () const { return data_ + size _; }
113
114     void clear () {
115         delete [] data_;
116         data_ = new T{10};
117         size _ = 0;
118         capacity_ = 10;
119     }
120
121     T* data () { return data_; }
122     const T* data () const { return data_; }
123 };
124
125
126 class RadixSort {

```

```

127 private:
128     RadixSort() = delete;
129
130     template <typename SortKey>
131     static void stable_counting_sort(MyVector<Item>& vec, MyVector<Item
132         >& buffer, SortKey key) {
133         int max_k = 0;
134         for (std::size_t i = 0; i != vec.size(); ++i) {
135             int k = static_cast<int>(key(vec[i]));
136             if (k > max_k) {
137                 max_k = k;
138             }
139
140             int count[max_k + 1]{0};
141             for (std::size_t i = 0; i != vec.size(); ++i) {
142                 ++count[static_cast<std::size_t>(key(vec[i]))];
143             }
144
145             for (int i = 1; i != max_k + 1; ++i) {
146                 count[i] += count[i - 1];
147             }
148
149             for (std::size_t i = vec.size(); i != 0; --i) {
150                 int k = static_cast<int>(key(vec[i - 1]));
151                 buffer[count[k] - 1] = vec[i - 1];
152                 --count[k];
153             }
154         }
155
156     public:
157     static void stable_radix_sort(MyVector<Item>& vec) {
158         if (vec.size() == 0) {
159             return;
160         }
161
162         MyVector<Item> buffer{vec.size()};
163         stable_counting_sort(vec, buffer, [](const Item& itm) -> uint8
164             _t { return itm.day; });
165         stable_counting_sort(buffer, vec, [](const Item& itm) -> uint8
166             _t { return itm.month; });
167         stable_counting_sort(vec, buffer, [](const Item& itm) -> uint16
168             _t { return itm.year; });

```

```

166
167         vec = std :: move(buffer);
168     }
169 };
170
171
172 std :: size _t find(const MyVector<char>& line, char c, std :: size _t start
    = 0) {
173     for (std :: size _t i = start; i != line.size(); ++i) {
174         if (line[i] == c) {
175             return i;
176         }
177     }
178
179     return -1;
180 }
181
182
183 MyVector<char> substr(const MyVector<char>& line, std :: size _t start,
    std :: size _t end) {
184     MyVector<char> result;
185     for (std :: size _t i = start; i != start + end; ++i) {
186         result.push_back(line[i]);
187     }
188
189     return result;
190 }
191
192
193 int stoi(const MyVector<char>& line) {
194     int result = 0;
195     for (std :: size _t i = 0; i != line.size(); ++i) {
196         result = result * 10 + (line[i] - '0');
197     }
198
199     return result;
200 }
201
202
203 Item parser(const MyVector<char>& line, std :: size _t ind) {
204     std :: size _t pos1 = find(line, '.');
205     std :: size _t pos2 = find(line, '.', pos1 + 1);
206     std :: size _t pos3 = find(line, '\\t');

```

```

207
208     uint8_t day = static_cast<uint8_t>(stoi(substr(line, 0, pos1)));
209     uint8_t month = static_cast<uint8_t>(stoi(substr(line, pos1 + 1,
210         pos2 - pos1 - 1)));
211     uint16_t year = static_cast<uint16_t>(stoi(substr(line, pos2 + 1,
212         pos3 - pos2 - 1)));
213
214     uint32_t pos = static_cast<uint32_t>(ind);
215     uint32_t len = static_cast<uint32_t>(line.size());
216
217
218     return Item{day, month, year, pos, len};
219 }
220
221
222 int main() {
223     MyVector<Item> vec;
224     MyVector<char> all_data;
225     uint32_t pos = 0;
226
227     char c;
228     MyVector<char> line;
229     bool line_feed_flag = false;
230     while (std::cin.get(c)) {
231         if (c == '\n') {
232             if (line.size() > 0) {
233                 vec.push_back(parser(line, pos));
234                 line.clear();
235                 pos = all_data.size();
236             }
237         } else {
238             all_data.push_back(c);
239             line.push_back(c);
240         }
241     }
242
243     if (line.size() > 0) {
244         vec.push_back(parser(line, pos));
245         line.clear();
246     }
247
248     RadixSort::stable_radix_sort(vec);
249
250     for (auto& elem : vec) {

```



```

248         std :: cout . write (all_data.data() + elem.pos, elem.len);
249         std :: cout  <<  "\n";
250     }
251
252     return 0;
253 }

```

6 Тесты

Входные данные:

```

15.03.22 12345
01.01.23 7890
07.11.21 42
31.12.22 999999
22.02.22 777
09.05.23 2023
13.06.21 100500
25.10.22 8
03.07.23 55555
18.08.21 13

```

Выходные данные:

```

13.06.21      100500
18.08.21      13
07.11.21      42
22.02.22      777
15.03.22      12345
25.10.22      8
31.12.22      999999
01.01.23      7890
09.05.23      2023
03.07.23      55555

```

7 Вывод

Асимптотика по времени в среднем составляет $O(n + k) * m$, где n - количество элементов, k - диапазон значений ключа, m - число разрядов (равно 3). Таким образом, асимптотика составляет $O(n + k)$. Асимптотика по памяти: $O(n)$, где n — число элементов.